A Multi-Source Data Integration Pipeline for Heterogeneous Student Records

Bhavya Sri
Department of Computer Science and Engineering
Hyderabad, India

Sahasra Kona
Department of Computer Science and Engineering
Hyderabad, India

Navyasri
Department of Computer Science and Engineering
Hyderabad, India

Abstract—Student information is often distributed across spreadsheets, relational databases, REST services, and externally maintained CSV files. These sources may describe the same entities while using different field names, formats, and data-quality conventions. This paper presents a modular Python-based Extract, Transform, Load (ETL) pipeline for extracting student records from heterogeneous sources, mapping them to a common schema, cleaning incomplete and duplicate entries, and integrating records using source-priority conflict resolution. The implementation uses Pandas for data processing, SQLAlchemy and PyMySQL for database connectivity, Flask for a local mock API, and Streamlit with Plotly for interactive analysis. In the supplied execution output, 39 records were reduced to 36 integrated records. The quality report records a reduction from eight missing values, two duplicates, and one invalid record to zero for each reported quality indicator after cleaning. The resulting dataset contains three represented sources and five course categories. The integrated output is explored through a dashboard containing summary indicators, course distributions, source contributions, and age statistics.

Keywords—ETL, data integration, heterogeneous data, data cleaning, schema mapping, Pandas, MySQL, Streamlit, data quality.

I. INTRODUCTION

Student information in practical computing environments is commonly maintained across independent systems. Academic records may originate in Excel workbooks, database tables, API responses, and externally maintained CSV files. Although these sources describe similar entities, their schemas and conventions may differ. A student identifier may be represented using different column names, while course names may appear as full department names or abbreviations. Directly combining these records can introduce inconsistent columns, duplicate entities, missing attributes, and unreliable summaries.

The project addresses this problem through a modular ETL workflow. Each source is extracted independently, mapped to a shared student schema, cleaned using explicit validation rules, and integrated into a consolidated dataset. The final data can be saved as CSV and, when configured, loaded into MySQL. A Streamlit dashboard provides summary indicators and interactive charts for exploring the integrated records.

The implementation contributes a set of source-specific extraction modules, schema mappings, data-quality operations, priority-based conflict resolution for repeated student identifiers, and a dashboard for inspecting the integrated output.

II. RELATED WORK

A. ETL and Data Integration

ETL systems provide a structured approach for moving data from operational sources into a consistent analytical representation. Data-frame libraries support tabular transformation and missing-value handling, while database abstraction libraries provide a common interface for relational operations. REST APIs enable structured data exchange between services, and dashboard frameworks simplify the presentation of analytical results. This project combines these established technologies in a reproducible student-data integration workflow. It focuses on practical schema reconciliation and data-quality processing rather than proposing a new ETL algorithm.

B. Data Quality and Visualization

Data quality operations such as duplicate removal, type conversion, validation, and normalization are important before records from multiple sources are combined. Visualization complements these operations by making source contributions, category distributions, and data characteristics easier to inspect. The proposed pipeline connects these stages in a single workflow.

III. PROPOSED SYSTEM

A. System Architecture

The system follows a layered architecture comprising data sources, extraction, transformation, cleaning, integration, loading, and visualization. The configured source modules include Excel, MySQL, REST API, and web/CSV input. Each extraction module returns a Pandas DataFrame. The transformation layer maps source-specific columns to the common fields student_id, name, age, course, and email, while retaining source information for integration and reporting.

After cleaning, records are grouped by student identifier. Conflicting records are resolved using the configured source priority: MySQL, API, Excel, and Web/CSV. The highest-priority record is retained as the base record, and missing or placeholder attributes may be filled from lower-priority records. The integrated dataset is then sorted by student identifier and written to the output CSV. MySQL loading is supported as an optional final step.

B. Data Sources

The supplied project package contains an Excel workbook, a web/CSV file, a Flask-based mock REST API, and a MySQL extraction module with SQL schema and sample-data scripts. The supplied integrated output represents three populated sources—Excel, API, and web/CSV—with 36 final records. MySQL is part of the implemented architecture but is not represented among the three populated sources in this particular output.

C. Common Data Model

Source-specific attributes are converted into a common structure consisting of student_id, name, age, course, email, and source. The transformation module includes mappings for spreadsheet labels, database column names, camelCase API fields, and web/CSV column names. Course labels are normalized to standard abbreviations such as CSE, ECE, IT, EEE, and ME.

IV. METHODOLOGY

A. Data Extraction

The extraction layer uses dedicated functions for each source. The Excel loader reads the workbook, the API loader requests JSON records from the local Flask service, the web loader reads the CSV file, and the MySQL loader obtains records through the database connection configuration. Each source is handled independently so that a failure in one source can be recorded without necessarily preventing extraction from the remaining sources.

B. Schema Transformation

After extraction, column mappings rename source-specific fields to the common schema. Course labels are normalized to abbreviations such as CSE, ECE, IT, EEE, and ME. This mapping reduces differences caused by alternate department names and makes category-wise analysis consistent across the integrated dataset.

C. Data Cleaning

The cleaning module records quality indicators, removes rows without a usable student identifier, and removes exact duplicate rows. Missing names and courses are replaced with "Unknown"; missing email values are replaced with a defined placeholder. Ages are converted to numeric values, missing or out-of-range ages are replaced with the median of valid ages, names are converted to title case, emails are lowercased, and ages are stored as integers. The implemented validity range for age is 16 to 60 years.

D. Record Integration and Conflict Resolution

Transformed source frames are combined and assigned a priority based on source. Records sharing a student_id are processed together. The highest-priority record becomes the base record; missing or placeholder values in its name, age, course, or email fields are filled from available lower-priority records. This method preserves the preferred source while recovering useful attributes where the preferred record is incomplete.

E. Loading and Visualization

The integrated data is saved to data/output/integrated_students.csv, and a text quality report is generated. The loader module supports writing the final data to MySQL when database configuration is available. The Streamlit dashboard reads the CSV output first and can fall back to MySQL. It provides filters for source and course, a name/email search field, an age-range control, KPI metrics, and Plotly charts.

V. IMPLEMENTATION

A. Development Environment

The implementation uses Python 3.10 or later. Pandas is used for DataFrame operations; OpenPyXL supports Excel input; Requests handles HTTP communication; Flask provides the mock API; SQLAlchemy and PyMySQL support MySQL connectivity; Streamlit builds the dashboard; Plotly renders interactive charts; python-dotenv loads configuration; and pytest is used for automated tests.

B. Modular Design

The source code separates extraction, transformation, cleaning, loading, orchestration, dashboard, and tests. The DataPipeline orchestrator coordinates extraction, transformation, cleaning, integration, optional MySQL loading, and output generation. The command-line entry point supports a --skip-mysql option, allowing the CSV workflow to run without an active MySQL server.

C. Testing

The project includes pytest modules for cleaning, transformation, and pipeline/extraction behavior. These tests are intended to check the main data-processing components. A test pass rate is not reported here because no test execution result was included in the supplied execution output.

VI. RESULTS AND DISCUSSION

The supplied quality report records 39 records before cleaning and 36 records after cleaning. Before cleaning, the report identifies eight missing values, two duplicate records, and one invalid record. After the cleaning stage, each of these reported quality indicators is zero. The integrated CSV contains 36 rows and six columns: student_id, name, age, course, email, and source. No missing values are present in the saved output.

The source distribution in the integrated output is 15 Excel records, 10 API records, and 11 web/CSV records. The course distribution is 13 CSE, 7 ECE, 7 IT, 5 ME, and 4 EEE records. The mean age is approximately 21.1 years, with observed ages ranging from 19 to 25 years. These values are computed from the supplied integrated CSV.

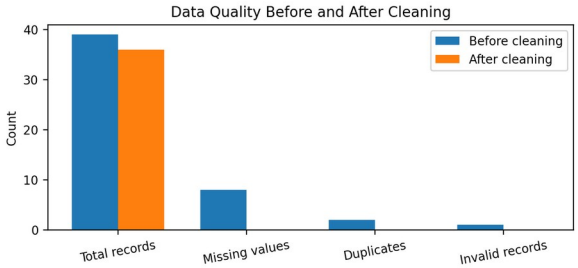


Fig. 1. Data-quality indicators before and after cleaning.

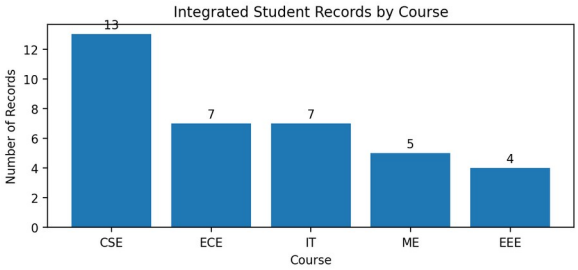


Fig. 2. Distribution of integrated records across courses.

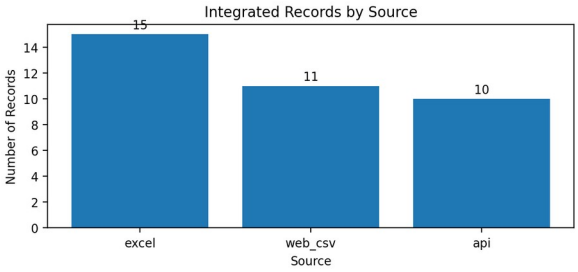


Fig. 3. Number of retained records by source.

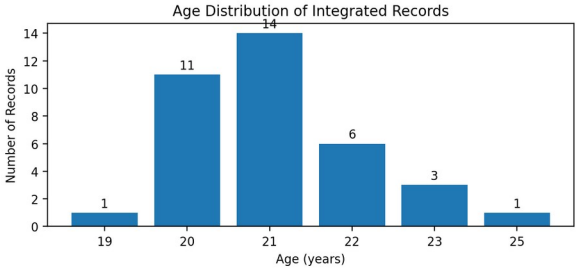


Fig. 4. Age distribution in the integrated dataset.

The results indicate that schema mapping and cleaning produced a consistent output suitable for dashboard analysis. The dashboard screenshot supplied with the project displays the same high-level totals: 36 records, three data sources, five courses, and an average age of 21.1. The charts make source contribution and course distribution easier to inspect than a

raw table alone. However, the current experiment uses a small project dataset; the results should not be interpreted as a benchmark of throughput or scalability on large institutional datasets.

VII. LIMITATIONS AND FUTURE ENHANCEMENTS

The demonstrated dataset is limited in size and represents only three populated sources in the saved output. The MySQL extraction and loading components require a separately configured database and were not demonstrated by the supplied output. The current conflict-resolution strategy is based on a fixed source priority and student identifier; it does not perform probabilistic entity matching when identifiers disagree. Future work may include larger real-world datasets, configurable validation rules, fuzzy entity resolution, incremental extraction, pipeline scheduling, richer audit logs, role-based dashboard access, and performance evaluation under higher loads.

VIII. CONCLUSION

This paper presented a Python-based multi-source student data integration pipeline that combines heterogeneous records through extraction, schema transformation, data cleaning, priority-based integration, and visualization. The supplied run reduced 39 input records to 36 integrated records and reported zero missing values, duplicates, and invalid records after cleaning. The resulting dataset consolidates three represented sources into a common six-column structure and is explored through a Streamlit dashboard. The project demonstrates a practical and modular approach to preparing heterogeneous student records for analysis, while leaving larger-scale validation and full MySQL deployment as areas for future work.

REFERENCES

[1] The pandas development team, “pandas documentation.” [Online]. Available: https://pandas.pydata.org/docs/.

[2] SQLAlchemy authors, “SQLAlchemy documentation.” [Online]. Available: https://docs.sqlalchemy.org/.

[3] Streamlit, “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io/.

[4] Plotly Technologies Inc., “Plotly Python graphing library.” [Online]. Available: https://plotly.com/python/.

[5] pytest development team, “pytest documentation.” [Online]. Available: https://docs.pytest.org/.

[6] Pallets Projects, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com/.